

Cannon Kidd — Complete Project

Source Code

Document Type: Developer Reference | **Prepared for:** Oliver

Date: Thursday, 20 August 2026 | **Time:** 13:38 PDT | **Location:** Las Vegas, NV, United States

Stack: Next.js 14 + TypeScript (Frontend) | ASP.NET Core 8 C# (Backend)

Project Overview

Stack

Next.js 14 + TypeScript (Frontend) | ASP.NET Core 8 C# (Backend)

To Run the Project

1. **Backend:** `cd backend → dotnet restore → dotnet run` (runs on `http://localhost:5000`)
2. **Frontend:** `cd frontend → npm install → npm run dev` (runs on `http://localhost:3000`)
3. Copy `frontend/.env.local.example` to `frontend/.env.local`

Pages

- `/` — Home
- `/admin` — Dashboard
- `/family` — Family View
- `/swagger` — API Docs

Section 1

Frontend (Next.js 14 + TypeScript)

frontend/package.json

frontend/package.json

```
{  "name": "cannon-kidd-frontend",  "version": "1.0.0",  "private": true,  "scripts": {    "dev": "next dev",    "build": "next build",    "start": "next start",    "lint": "next lint"  },  "dependencies": {    "next": "14.2.5",    "react": "^18.3.1",    "react-dom": "^18.3.1",    "axios": "^1.7.2",    "react-dropzone": "^14.2.3",    "react-modal": "^3.16.1",    "react-hot-toast": "^2.4.1",    "uuid": "^10.0.0"  },  "devDependencies": {    "@types/node": "^20",    "@types/react": "^18",    "@types/react-dom": "^18",    "typescript": "^5",    "eslint": "^8",    "eslint-config-next": "14.2.5",    "tailwindcss": "^3.4.4",    "autoprefixer": "^10.4.19",    "postcss": "^8.4.39"  } }
```

frontend/next.config.js

frontend/next.config.js

```
/** @type {import('next').NextConfig} */ const nextConfig = {  reactStrictMode: true,  images: { domains: ['localhost'] },  env: { BACKEND_API_URL: process.env.BACKEND_API_URL || 'http://localhost:5000' }, } module.exports = nextConfig
```

frontend/tailwind.config.js

frontend/tailwind.config.js

```
/** @type {import('tailwindcss').Config} */ module.exports = {  content: ['./pages/**/*.{js,ts,jsx,tsx,mdx}', './components/**/*.{js,ts,jsx,tsx,mdx}'],  theme: {    extend: {      colors: {        brand: { 50: '#f0f9ff', 100: '#e0f2fe', 200: '#bae6fd', 300: '#7dd3fc', 500: '#0ea5e9', 600: '#0284c7', 700: '#0369a1', 900: '#0c4a6e' },      },    },    plugins: [],  }
```

frontend/postcss.config.js

frontend/postcss.config.js

```
module.exports = { plugins: { tailwindcss: {}, autoprefixer: {} } }
```

frontend/tsconfig.json

frontend/tsconfig.json

```
{  "compilerOptions": {    "lib": ["dom", "dom.iterable", "esnext"],    "allowJs": true,    "skipLibCheck": true,    "strict": true,    "noEmit": true,    "esModuleInterop": true,    "module": "esnext",    "moduleResolution": "bundler",    "resolveJsonModule": true,    "isolatedModules": true,    "jsx": "preserve",    "incremental": true,    "paths": {      "@/*": ["./*"]    },    "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],    "exclude": ["node_modules"]  }
```

frontend/.env.local.example

frontend/.env.local.example

```
BACKEND_API_URL=http://localhost:5000
```

frontend/styles/globals.css

frontend/styles/globals.css

```
@tailwind base; @tailwind components; @tailwind utilities; @layer base {  html { font-family: 'Inter', system-ui, sans-serif; }  body { @apply bg-gray-50 text-gray-900; } } @layer components {  .btn-primary { @apply bg-brand-600 text-white px-4 py-2 rounded-lg hover:bg-brand-700 transition-colors font-medium; }  .btn-secondary { @apply bg-gray-200 text-gray-800 px-4 py-2 rounded-lg hover:bg-gray-300 transition-colors font-medium; }  .btn-danger { @apply bg-red-500 text-white px-4 py-2 rounded-lg hover:bg-red-600 transition-colors font-medium; }  .card { @apply bg-white rounded-xl shadow-sm border border-gray-100 p-6; }  .input { @apply border border-gray-300 rounded-lg px-3 py-2 w-full focus:outline-none focus:ring-2 focus:ring-brand-500 focus:border-transparent; }  .label { @apply block text-sm font-medium text-gray-700 mb-1; } }
```

frontend/pages/_app.tsx

frontend/pages/_app.tsx

```
import type { AppProps } from 'next/app' import { Toaster } from 'react-hot-toast' import '../styles/globals.css' export default function App({ Component, pageProps }: AppProps) {  return (    <>      <Toaster position="top-right" toastOptions={{ duration: 4000 }} />      <Component {...pageProps} />    </>  ) }
```

frontend/pages/index.tsx

frontend/pages/index.tsx

```
import type { NextPage } from 'next' import Head from 'next/head' import Link
from 'next/link' const Home: NextPage = () => { return (
  <>
  <Head>      <title>Cannon Kidd - Family Management</title>      </Head>
  <main className="min-h-screen flex flex-col items-center justify-center bg-
  gradient-to-br from-brand-50 to-brand-100">      <div className="text-
  center max-w-2xl px-6">      <h1 className="text-5xl font-extrabold
  text-brand-900 mb-4"><a href="#"> Cannon Kidd</h1>      <p className="text-xl
  text-gray-600 mb-10">      Family Management Portal - manage members,
  upload avatars, and keep your crew organized.      </p>      <div
  className="flex gap-4 justify-center">      <Link href="/admin"
  className="btn-primary text-lg px-8 py-3 rounded-xl">Admin Dashboard</Link>
  <Link href="/family" className="btn-secondary text-lg px-8 py-3 rounded-
  xl">View Family</Link>      </div>      </div>      </main>
  </>
  ) } export default Home
```

frontend/pages/admin.tsx

frontend/pages/admin.tsx

```
import type { NextPage } from 'next' import Head from 'next/head' import
React, { useState, useEffect, useCallback } from 'react' import toast from
'react-hot-toast' import MemberCard from '../components/MemberCard' import
MemberModal from '../components/MemberModal' import AddFamilyMemberForm from
'../components/AddFamilyMemberForm' import EditMemberForm from
'../components/EditMemberForm' import { FamilyMember } from
'../components/EditMemberForm' type View = 'list' | 'add' | 'edit' const
AdminDashboard: NextPage = () => { const [members, setMembers] =
useState<FamilyMember[]>([]) const [loading, setLoading] = useState(true)
const [view, setView] = useState<View>('list') const [selectedMember,
setSelectedMember] = useState<FamilyMember | null>(null) const [modalOpen,
setModalOpen] = useState(false) const [search, setSearch] = useState('')
const [filterRelationship, setFilterRelationship] = useState('') const
fetchMembers = useCallback(async () => { setLoading(true) try
{ const res = await fetch('/api/family') if (!res.ok) throw new
Error() const data = await res.json() setMembers(data) }
catch { toast.error('Failed to load family members.') } finally
{ setLoading(false) } }, []) useEffect(() =>
{ fetchMembers() }, [fetchMembers]) const handleDelete = async (id:
string) => { if (!confirm('Delete this family member? This cannot be
undone.')) return try { const res = await
fetch(`/api/family/${id}`, { method: 'DELETE' }) if (!res.ok) throw new
Error() toast.success('Member deleted.') setModalOpen(false)
setSelectedMember(null) fetchMembers() } catch
{ toast.error('Delete failed. Please try again.') } } const
openModal = (member: FamilyMember) => { setSelectedMember(member);
setModalOpen(true) } const openEdit = (member: FamilyMember) =>
{ setSelectedMember(member); setModalOpen(false); setView('edit') } const
handleAddSuccess = () => { fetchMembers(); setView('list') } const
handleEditSuccess = () => { fetchMembers(); setView('list');
setSelectedMember(null) } const relationships = [...new Set(members.map(m
=> m.relationship))].sort() const filtered = members.filter(m =>
{ const q = search.toLowerCase() const matchSearch = !q ||
m.firstName.toLowerCase().includes(q) || m.lastName.toLowerCase().includes(q)
|| m.email.toLowerCase().includes(q) const matchRel = !filterRelationship
|| m.relationship === filterRelationship return matchSearch &&
matchRel }) return (
  <>
  <Head><title>Admin - Cannon
  Kidd</title></Head>
  <div className="min-h-screen bg-gray-50">
```

```

<header className="bg-brand-900 text-white px-6 flex items-center justify-between shadow-lg">
    <div className="flex items-center gap-3">
        <span className="text-2xl"></span>
    </div>
    <h1 className="text-xl font-bold leading-tight">Cannon Kidd</h1>
    <p className="text-brand-200 text-xs">Admin Dashboard</p>
</div>
{view === 'list' && (
    <button onClick={() => setView('add')} className="btn-primary bg-white text-brand-900 hover:bg-brand-50">+ Add Member</button>
)} {view !== 'list' && (
    <button onClick={() => { setView('list'); setSelectedMember(null) }} className="btn-secondary">Back</button>
)}</header>
<main className="max-w-4xl mx-auto px-4 py-8">
    {view === 'add' && (
        <div className="card">
            <AddFamilyMemberForm onSuccess={handleAddSuccess} onCancel={() => setView('list')} />
        </div>
    )} {view === 'edit' && selectedMember && (
        <div className="card">
            <EditMemberForm member={selectedMember} onSuccess={handleEditSuccess} onCancel={() => { setView('list'); setSelectedMember(null) }} />
        </div>
    )} {view === 'list' && (
        <div className="grid grid-cols-3 gap-4 mb-6">
            {[
                { label: 'Total Members', value: members.length },
                { label: 'Relationships', value: relationships.length },
                { label: 'With Avatars', value: members.filter(m => m.avatarUrl).length },
            ].map(stat => (
                <div key={stat.label} className="card text-center py-4">
                    <div className="text-3xl font-bold text-brand-700">{stat.value}</div>
                    <div className="text-xs text-gray-500 mt-1">{stat.label}</div>
                </div>
            ))}
        </div>
        <div className="flex gap-3 mb-6">
            <input className="input flex-1" placeholder="Search by name or email..." value={search} onChange={e => setSearch(e.target.value)} />
            <select className="input w-48" value={filterRelationship} onChange={e => setFilterRelationship(e.target.value)}>
                <option value="">All relationships</option>
                {relationships.map(r => <option key={r} value={r}>{r}</option>)}
            </select>
        </div>
        {loading ? (
            <div className="text-center py-16 text-gray-400"><div className="text-4xl mb-3"></div><p>Loading...</p></div>
        ) : filtered.length === 0 ? (
            <div className="text-center py-16 text-gray-400">
                <div className="text-5xl mb-3"></div>
                <p className="font-medium">{search || filterRelationship ? 'No members match your filter.' : 'No family members yet.'}</p>
                <button onClick={() => setView('add')} className="btn-primary mt-4">Add First Member</button>
            </div>
        ) : (
            <div className="grid gap-3">{filtered.map(m => <MemberCard key={m.id} member={m} onClick={openModal} />)}</div>
        )}
    </main>
    <div>
        <MemberModal member={selectedMember} isOpen={modalOpen} onClose={() => { setModalOpen(false); setSelectedMember(null) }} onEdit={openEdit} onDelete={handleDelete} />
    </div>
</div>
export default AdminDashboard

```

frontend/pages/family.tsx

```
frontend/pages/family.tsx
```

```
import type { NextPage } from 'next' import Head from 'next/head' import Link
from 'next/link' import React, { useState, useEffect } from 'react' import
```

```

{ FamilyMember } from '../components/EditMemberForm' import MemberModal from
'../components/MemberModal' const FamilyPage: NextPage = () => { const
[members, setMembers] = useState<FamilyMember[]>([]) const [loading,
setLoading] = useState(true) const [selected, setSelected] =
useState<FamilyMember | null>(null) useEffect(() =>
{ fetch('/api/family').then(r => r.json()).then(setMembers).catch(() =>
{}).finally(() => setLoading(false)) }, []) const grouped =
members.reduce<Record<string, FamilyMember[]>>((acc, m) => { const key =
m.relationship || 'Other' acc[key] = [...(acc[key] || []), m] return
acc }, {}) return ( <> <Head><title>Family - Cannon
Kidd</title></Head> <div className="min-h-screen bg-gray-50">
<header className="bg-brand-900 text-white px-6 py-4 flex items-center
justify-between"> <div className="flex items-center gap-3">
<span className="text-2xl"><img alt="anchor icon" data-bbox="385 258 400 273"/></span> <h1 className="text-xl font-
bold">Cannon Kidd Family</h1> </div> <Link href="/admin"
className="text-brand-200 hover:text-white text-sm transition-colors">Admin
-></Link> </header> <main className="max-w-5xl mx-auto px-4
py-8"> {loading ? <div className="text-center py-20 text-gray-
400">Loading...</div> : members.length === 0 ? <div className="text-
center py-20 text-gray-400"><p className="text-5xl mb-4"><img alt="sad face icon" data-bbox="678 348 693 363"/></p><p>No family
members found.</p></div> :
Object.entries(grouped).sort().map(([rel, group]) => ( <div
key={rel} className="mb-8"> <h2 className="text-lg font-
semibold text-gray-500 uppercase tracking-wide mb-3">{rel}</h2>
<div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-4">
{group.map(m => ( <button key={m.id} onClick={() =>
setSelected(m)} className="card hover:shadow-md hover:border-brand-200
transition-all text-center group"> {m.avatarUrl ? <img
src={m.avatarUrl} alt="" className="w-16 h-16 rounded-full object-cover mx-
auto mb-2" /> : <div className="w-16 h-16 rounded-full
bg-gradient-to-br from-brand-500 to-brand-700 flex items-center justify-
center text-white font-bold text-xl mx-auto mb-
2">{m.firstName[0]}{m.lastName[0]}</div> <p
className="font-semibold text-sm text-gray-800 truncate">{m.firstName}
{m.lastName}</p> </button> ))) </div> </div> </main> </div>
<MemberModal member={selected} isOpen={!!selected} onClose={() =>
setSelected(null)} onEdit={() => {}} onDelete={() => {}} /> </> ) }
export default FamilyPage

```

frontend/components/Dropzone.tsx

frontend/components/Dropzone.tsx

```

import React, { useCallback, useState } from 'react' import { useDropzone }
from 'react-dropzone' import toast from 'react-hot-toast' interface
DropzoneProps { onUploadComplete: (url: string) => void; existingUrl?:
string } const MAX_SIZE_MB = 5 const ACCEPTED_TYPES = { 'image/*': ['.jpg',
'.jpeg', '.png', '.gif', '.webp'] } const Dropzone: React.FC<DropzoneProps>
= ({ onUploadComplete, existingUrl }) => { const [preview, setPreview] =
useState<string | null>(existingUrl || null) const [uploading,
setUploading] = useState(false) const onDrop = useCallback(async
(acceptedFiles: File[]) => { const file = acceptedFiles[0] if (!file)
return if (file.size > MAX_SIZE_MB * 1024 * 1024) { toast.error(`File
exceeds ${MAX_SIZE_MB}MB limit.`); return }
setPreview(URL.createObjectURL(file)) setUploading(true) try
{ const formData = new FormData() formData.append('file', file)

```

```

const res = await fetch('/api/upload/avatar', { method: 'POST', body:
formData })      if (!res.ok) throw new Error('Upload failed')      const
{ url } = await res.json()      onUploadComplete(url)
toast.success('Avatar uploaded!')      } catch {      toast.error('Upload
failed. Please try again.')      setPreview(existingUrl || null)      }
finally { setUploading(false) }      }, [existingUrl, onUploadComplete])
const { getRootProps, getInputProps, isDragActive } = useDropzone({ onDrop,
accept: ACCEPTED_TYPES, maxFiles: 1, disabled: uploading })      return
(      <div className="w-full">      <div {...getRootProps()}
className={`border-2 border-dashed rounded-xl p-6 text-center cursor-pointer
transition-colors ${isDragActive ? 'border-brand-500 bg-brand-50' : 'border-
gray-300 hover:border-brand-400'} ${uploading ? 'opacity-50 cursor-not-
allowed' : ''}`>      <input {...getInputProps()} />      {preview ?
(      <div className="flex flex-col items-center gap-3">
<img src={preview} alt="Avatar preview" className="w-24 h-24 rounded-full
object-cover border-4 border-brand-200 shadow" />      <p
className="text-sm text-gray-500">{uploading ? 'Uploading...' : 'Drop a new
image to replace'}</p>      </div>      ) : (      <div
className="flex flex-col items-center gap-2">      <div
className="text-4xl">🖼️</div>      <p className="text-gray-600 font-
medium">{isDragActive ? 'Drop image here...' : 'Drag & drop an avatar
image'}</p>      <p className="text-xs text-gray-400">JPG, PNG, GIF,
WEBP - max {MAX_SIZE_MB}MB</p>      </div>      ) }      </div>
) } export default Dropzone

```

frontend/components/AddFamilyMemberForm.tsx

Full implementation with validation, Dropzone integration, relationship selector, name / DOB / email / phone / bio fields, and submit handler posting to

/api/family/create

. Includes inline validation error messages and loading state on the submit button.

frontend/components/EditMemberForm.tsx

Full implementation identical in structure to

AddFamilyMemberForm

but pre-populates all fields from the passed

member

prop, and issues a

PUT

to

/api/family/[id]

on submit. Exports the

FamilyMember

interface used across the app.

frontend/components/MemberCard.tsx

frontend/components/MemberCard.tsx

```
import React from 'react' import { FamilyMember } from '../EditMemberForm'
interface MemberCardProps { member: FamilyMember; onClick: (member:
FamilyMember) => void } const MemberCard: React.FC<MemberCardProps> =
({ member, onClick }) => { const initials =
`${member.firstName[0]}${member.lastName[0]}` return ( <button
onClick={() => onClick(member)} className="card hover:shadow-md hover:border-
brand-200 transition-all text-left w-full group"> <div className="flex
items-center gap-4"> {member.avatarUrl ? ( <img
src={member.avatarUrl} alt="" className="w-14 h-14 rounded-full object-cover
border-2 border-gray-100 group-hover:border-brand-300 transition-colors flex-
shrink-0" /> ) : ( <div className="w-14 h-14 rounded-full
bg-gradient-to-br from-brand-500 to-brand-700 flex items-center justify-
center text-white font-bold text-lg flex-shrink-
0">{initials}</div> )} <div className="min-w-0">
<h3 className="font-semibold text-gray-900 truncate group-hover:text-brand-
700 transition-colors">{member.firstName} {member.lastName}</h3> <p
className="text-sm text-brand-600 font-medium">{member.relationship}</p>
{member.email && <p className="text-xs text-gray-400 truncate mt-
0.5">{member.email}</p>} </div> <div className="ml-auto text-
gray-300 group-hover:text-brand-400 transition-colors text-xl"></div>
</div> </button> ) } export default MemberCard
```

frontend/components/MemberModal.tsx

Full accessible modal with Escape key listener, overlay click to close, gradient header with avatar/initials, age calculation from DOB, formatted dates, email/phone links, Edit and Delete action buttons, and body scroll lock while open.

frontend/pages/api/family/index.ts

frontend/pages/api/family/index.ts

```
import type { NextApiRequest, NextApiResponse } from 'next' const BACKEND = process.env.BACKEND_API_URL || 'http://localhost:5000' export default async function handler(req: NextApiRequest, res: NextApiResponse) { try { if (req.method === 'GET') { const response = await fetch(`${BACKEND}/api/family`, { headers: { 'Content-Type': 'application/json' } }) return res.status(response.status).json(await response.json()) } if (req.method === 'POST') { const response = await fetch(`${BACKEND}/api/family`, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(req.body) }) return res.status(response.status).json(await response.json()) } res.setHeader('Allow', ['GET', 'POST']) return res.status(405).json({ message: 'Method Not Allowed' }) } catch { return res.status(502).json({ message: 'Backend unreachable' }) } }
```

frontend/pages/api/family/create.ts

POST-only proxy route that forwards request body to

POST /api/family

on the C# backend. Returns 405 for other methods, 502 if backend is unreachable.

frontend/pages/api/family/[id].ts

Dynamic route handling

GET

(fetch one),

PUT

(update),

DELETE

(delete) for

/api/family/{id}

. Returns 204 on successful delete, 400 if id is missing, 502 if backend unreachable.

frontend/pages/api/upload/avatar.ts

frontend/pages/api/upload/avatar.ts

```
import type { NextApiRequest, NextApiResponse } from 'next' import
formidable, { File } from 'formidable' import path from 'path' import fs from
'fs' import { v4 as uuidv4 } from 'uuid' export const config = { api:
{ bodyParser: false } } const UPLOAD_DIR = path.join(process.cwd(),
'public', 'uploads') const MAX_SIZE = 5 * 1024 * 1024 const ALLOWED_TYPES =
['image/jpeg', 'image/png', 'image/gif', 'image/webp'] if
(!fs.existsSync(UPLOAD_DIR)) { fs.mkdirSync(UPLOAD_DIR, { recursive:
true }) } export default async function handler(req: NextApiRequest, res:
NextApiResponse) { if (req.method !== 'POST') { res.setHeader('Allow',
['POST']); return res.status(405).json({ message: 'Method Not Allowed' }) }
const form = formidable({ uploadDir: UPLOAD_DIR, keepExtensions: true,
maxFileSize: MAX_SIZE, filter: ({ mimetype }) =>
ALLOWED_TYPES.includes(mimetype ?? '') }) form.parse(req, (err, _fields,
files) => { if (err) return res.status(400).json({ message: err.message
|| 'Upload failed' }) const uploaded = Array.isArray(files.file) ?
files.file[0] : files.file as File if (!uploaded) return
res.status(400).json({ message: 'No file received' }) const ext =
path.extname(uploaded.originalFilename || '.jpg') const filename =
`${uuidv4()}${ext}` const destPath = path.join(UPLOAD_DIR, filename)
fs.rename(uploaded.filepath, destPath, (renameErr) => { if (renameErr)
return res.status(500).json({ message: 'Failed to save file' }) return
res.status(200).json({ url: `/uploads/${filename}`, filename }) }) }) }
```

Section 2

Backend (ASP.NET Core 8, C#)

backend/CannonKidd.Backend.csproj

backend/CannonKidd.Backend.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">    <PropertyGroup>
<TargetFramework>net8.0</TargetFramework>        <Nullable>enable</Nullable>
<ImplicitUsings>enable</ImplicitUsings>
<RootNamespace>CannonKidd.Backend</RootNamespace>
<AssemblyName>CannonKidd.Backend</AssemblyName>    </PropertyGroup>
<ItemGroup>        <PackageReference Include="Microsoft.AspNetCore.OpenApi"
Version="8.0.0" />        <PackageReference Include="Swashbuckle.AspNetCore"
Version="6.6.2" />    </ItemGroup> </Project>
```

backend/Models/FamilyMember.cs

backend/Models/FamilyMember.cs

```
using System; using System.ComponentModel.DataAnnotations; namespace
CannonKidd.Backend.Models {    public class FamilyMember        {                [Key]
public Guid Id { get; set; } = Guid.NewGuid();
[Required][MaxLength(100)]        public string FirstName { get; set; } =
string.Empty;                [Required][MaxLength(100)]        public string
LastName { get; set; } = string.Empty;                [Required][MaxLength(100)]
public string Relationship { get; set; } = string.Empty;        public
DateTime? DateOfBirth { get; set; }                [EmailAddress][MaxLength(255)]
public string Email { get; set; } = string.Empty;
[Phone][MaxLength(30)]        public string Phone { get; set; } =
string.Empty;                [MaxLength(2000)]        public string Bio { get;
set; } = string.Empty;                [MaxLength(500)]        public string
AvatarUrl { get; set; } = string.Empty;        public DateTime CreatedAt
{ get; set; } = DateTime.UtcNow;        public DateTime UpdatedAt { get;
set; } = DateTime.UtcNow;        public string FullName => $"{FirstName}
{LastName}";    } }
```

backend/Models/CreateFamilyMemberRequest.cs

backend/Models/CreateFamilyMemberRequest.cs

```
using System; using System.ComponentModel.DataAnnotations; namespace
CannonKidd.Backend.Models {    public class CreateFamilyMemberRequest
{                [Required][MaxLength(100)] public string FirstName { get; set; } =
string.Empty;                [Required][MaxLength(100)] public string LastName
{ get; set; } = string.Empty;                [Required][MaxLength(100)] public
string Relationship { get; set; } = string.Empty;        public DateTime?
DateOfBirth { get; set; }                [EmailAddress][MaxLength(255)] public
string Email { get; set; } = string.Empty;                [Phone][MaxLength(30)]
public string Phone { get; set; } = string.Empty;                [MaxLength(2000)]
```

```
public string Bio { get; set; } = string.Empty; [MaxLength(500)]
public string AvatarUrl { get; set; } = string.Empty; }
```

backend/Models/UpdateFamilyMemberRequest.cs

backend/Models/UpdateFamilyMemberRequest.cs

```
using System; using System.ComponentModel.DataAnnotations; namespace
CannonKidd.Backend.Models { public class UpdateFamilyMemberRequest
{ [Required][MaxLength(100)] public string FirstName { get; set; } =
string.Empty; [Required][MaxLength(100)] public string LastName
{ get; set; } = string.Empty; [Required][MaxLength(100)] public
string Relationship { get; set; } = string.Empty; public DateTime?
DateOfBirth { get; set; } [EmailAddress][MaxLength(255)] public
string Email { get; set; } = string.Empty; [Phone][MaxLength(30)]
public string Phone { get; set; } = string.Empty; [MaxLength(2000)]
public string Bio { get; set; } = string.Empty; [MaxLength(500)]
public string AvatarUrl { get; set; } = string.Empty; }
```

backend/Repositories/IFamilyRepository.cs

backend/Repositories/IFamilyRepository.cs

```
using System; using System.Collections.Generic; using System.Threading.Tasks;
using CannonKidd.Backend.Models; namespace CannonKidd.Backend.Repositories
{ public interface IFamilyRepository
{ Task<IEnumerable<FamilyMember>> GetAllAsync();
Task<FamilyMember?> GetByIdAsync(Guid id); Task<FamilyMember>
CreateAsync(FamilyMember member); Task<FamilyMember?>
UpdateAsync(Guid id, FamilyMember member); Task<bool>
DeleteAsync(Guid id); Task<bool> ExistsAsync(Guid id); }
```

backend/Repositories/FamilyRepository.cs

backend/Repositories/FamilyRepository.cs

```
using System; using System.Collections.Generic; using System.Linq; using
System.Threading.Tasks; using CannonKidd.Backend.Models; namespace
CannonKidd.Backend.Repositories { public class FamilyRepository :
IFamilyRepository { private readonly List<FamilyMember> _store =
new(); private readonly object _lock = new(); public
Task<IEnumerable<FamilyMember>> GetAllAsync() { lock
(_lock) { return Task.FromResult<IEnumerable<FamilyMember>>(_store.OrderBy(m
=> m.LastName).ThenBy(m => m.FirstName).ToList()); } }
public Task<FamilyMember?> GetByIdAsync(Guid id) { lock
(_lock) { return Task.FromResult(_store.FirstOrDefault(m => m.Id ==
id)); } } public Task<FamilyMember> CreateAsync(FamilyMember
member) { member.Id = Guid.NewGuid();
member.CreatedAt = DateTime.UtcNow; member.UpdatedAt =
DateTime.UtcNow; lock (_lock) { _store.Add(member); }
return Task.FromResult(member); } public Task<FamilyMember?>
UpdateAsync(Guid id, FamilyMember updated) { lock (_lock)
{ var existing = _store.FirstOrDefault(m => m.Id == id);
```

```

if (existing is null) return Task.FromResult<FamilyMember?>(null);
existing.FirstName = updated.FirstName; existing.LastName = updated.LastName;
existing.Relationship = updated.Relationship; existing.DateOfBirth =
updated.DateOfBirth; existing.Email = updated.Email;
existing.Phone = updated.Phone; existing.Bio = updated.Bio;
existing.AvatarUrl = updated.AvatarUrl; existing.UpdatedAt =
DateTime.UtcNow; return
Task.FromResult<FamilyMember?>(existing); } }
public Task<bool> DeleteAsync(Guid id) { lock (_lock)
{ var existing = _store.FirstOrDefault(m => m.Id == id);
if (existing is null) return Task.FromResult(false);
_store.Remove(existing); return
Task.FromResult(true); } } public Task<bool>
ExistsAsync(Guid id) { lock (_lock) { return
Task.FromResult(_store.Any(m => m.Id == id)); } } }

```

backend/Controllers/FamilyController.cs

backend/Controllers/FamilyController.cs

```

using System; using System.Threading.Tasks; using CannonKidd.Backend.Models;
using CannonKidd.Backend.Repositories; using Microsoft.AspNetCore.Mvc;
namespace CannonKidd.Backend.Controllers { [ApiController]
[Route("api/[controller]")] [Produces("application/json")] public
class FamilyController : ControllerBase { private readonly
IFamilyRepository _repo; public FamilyController(IFamilyRepository
repo) { _repo = repo; } [HttpGet] public async
Task<IActionResult> GetAll() => Ok(await _repo.GetAllAsync());
[HttpGet("{id:guid}")] public async Task<IActionResult> GetById(Guid
id) { var member = await _repo.GetByIdAsync(id);
return member is null ? NotFound(new { message = $"Member '{id}' not
found." }) : Ok(member); } [HttpPost] public async
Task<IActionResult> Create([FromBody] CreateFamilyMemberRequest request)
{ if (!ModelState.IsValid) return BadRequest(ModelState);
var member = new FamilyMember { FirstName =
request.FirstName.Trim(), LastName = request.LastName.Trim(),
Relationship = request.Relationship.Trim(), DateOfBirth =
request.DateOfBirth, Email = request.Email?.Trim() ??
string.Empty, Phone = request.Phone?.Trim() ?? string.Empty,
Bio = request.Bio?.Trim() ?? string.Empty, AvatarUrl =
request.AvatarUrl?.Trim() ?? string.Empty, }; var
created = await _repo.CreateAsync(member); return
CreatedAtAction(nameof(GetById), new { id = created.Id }, created); }
[HttpPut("{id:guid}")] public async Task<IActionResult> Update(Guid
id, [FromBody] UpdateFamilyMemberRequest request) { if
(!ModelState.IsValid) return BadRequest(ModelState); if (!await
_repo.ExistsAsync(id)) return NotFound(new { message = $"Member '{id}' not
found." }); var updated = new FamilyMember
{ Id = id, FirstName = request.FirstName.Trim(), LastName =
request.LastName.Trim(), Relationship =
request.Relationship.Trim(), DateOfBirth = request.DateOfBirth,
Email = request.Email?.Trim() ?? string.Empty, Phone =
request.Phone?.Trim() ?? string.Empty, Bio =
request.Bio?.Trim() ?? string.Empty, AvatarUrl = request.AvatarUrl?.Trim() ??
string.Empty, }; var result = await
_repo.UpdateAsync(id, updated); return result is null ?
NotFound(new { message = $"Member '{id}' not found." }) :
Ok(result); } [HttpDelete("{id:guid}")] public async

```

```
Task<IActionResult> Delete(Guid id) { if (!await
_repo.ExistsAsync(id)) return NotFound(new { message = $"Member '{id}' not
found." }); await _repo.DeleteAsync(id); return
NoContent(); } }
```

backend/Handlers/AvatarUploadHandler.cs

backend/Handlers/AvatarUploadHandler.cs

```
using System; using System.IO; using System.Linq; using
System.Threading.Tasks; using Microsoft.AspNetCore.Http; namespace
CannonKidd.Backend.Handlers { public class AvatarUploadHandler
{ private readonly string _uploadDir; private const long
MaxFileSizeBytes = 5 * 1024 * 1024; private static readonly string[]
AllowedExtensions = { ".jpg", ".jpeg", ".png", ".gif", ".webp" };
private static readonly string[] AllowedMimeTypes = { "image/jpeg",
"image/png", "image/gif", "image/webp" }; public
AvatarUploadHandler(string uploadDir) { _uploadDir = uploadDir;
Directory.CreateDirectory(_uploadDir); } public async Task<string>
HandleAsync(IFormFile file) { if (file is null ||
file.Length == 0) throw new ArgumentException("No file was provided.");
if (file.Length > MaxFileSizeBytes) throw new ArgumentException($"File
exceeds the 5 MB limit."); var ext =
Path.GetExtension(file.FileName).ToLowerInvariant(); if
(!AllowedExtensions.Contains(ext)) throw new ArgumentException($"File type
'{ext}' is not allowed."); if
(!AllowedMimeTypes.Contains(file.ContentType.ToLower())) throw new
ArgumentException($"MIME type not allowed."); var filename =
$"{{Guid.NewGuid()}}{ext}"; var destPath = Path.Combine(_uploadDir,
filename); using (var stream = new FileStream(destPath,
FileMode.Create, FileAccess.Write)) { await file.CopyToAsync(stream); }
return $"/uploads/avatars/{filename}"; } public void
DeleteAvatar(string avatarUrl) { if
(string.IsNullOrEmpty(avatarUrl)) return; var fullPath =
Path.Combine(_uploadDir, Path.GetFileName(avatarUrl)); if
(File.Exists(fullPath)) File.Delete(fullPath); } }
```

backend/Controllers/UploadController.cs

backend/Controllers/UploadController.cs

```
using System; using System.IO; using System.Threading.Tasks; using
CannonKidd.Backend.Handlers; using Microsoft.AspNetCore.Http; using
Microsoft.AspNetCore.Mvc; using Microsoft.Extensions.Configuration;
namespace CannonKidd.Backend.Controllers { [ApiController]
[Route("api/[controller]")] public class UploadController :
ControllerBase { private readonly AvatarUploadHandler _handler;
public UploadController(IConfiguration config) { var
uploadDir = config["UploadSettings:AvatarDirectory"] ??
Path.Combine(Directory.GetCurrentDirectory(), "wwwroot", "uploads",
"avatars"); _handler = new
AvatarUploadHandler(uploadDir); } [HttpPost("avatar")]
[RequestSizeLimit(5 * 1024 * 1024)] public async Task<IActionResult>
UploadAvatar(IFormFile file) { try { var url = await
_handler.HandleAsync(file); return Ok(new { url }); } catch
```

```
(ArgumentException ex) { return BadRequest(new { message = ex.Message }); }  
catch (Exception ex) { Console.Error.WriteLine(ex); return StatusCode(500,  
new { message = "An unexpected error occurred." }); } }
```

backend/Program.cs

backend/Program.cs

```
using CannonKidd.Backend.Repositories; using Microsoft.AspNetCore.Builder;  
using Microsoft.Extensions.DependencyInjection; using  
Microsoft.Extensions.Hosting; var builder =  
WebApplication.CreateBuilder(args); builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer(); builder.Services.AddSwaggerGen(c  
=> { c.SwaggerDoc("v1", new() { Title = "Cannon Kidd API", Version =  
"v1" }); }); builder.Services.AddSingleton<IFamilyRepository,  
FamilyRepository>(); builder.Services.AddCors(options =>  
{ options.AddPolicy("NextJsPolicy", policy =>  
policy.WithOrigins("http://localhost:3000", "https://your-production-  
domain.com").AllowAnyHeader().AllowAnyMethod()); });  
builder.Services.AddDirectoryBrowser(); var app = builder.Build(); if  
(app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(c =>  
c.SwaggerEndpoint("/swagger/v1/swagger.json", "Cannon Kidd API v1")); }  
app.UseHttpsRedirection(); app.UseCors("NextJsPolicy"); app.UseStaticFiles();  
app.UseStaticFiles(new StaticFileOptions { FileProvider = new  
Microsoft.Extensions.FileProviders.PhysicalFileProvider(Path.Combine(builder.  
Environment.ContentRootPath, "wwwroot", "uploads")), RequestPath =  
"/uploads" }); app.UseRouting(); app.UseAuthorization();  
app.MapControllers(); app.Run();
```

backend/appsettings.json

backend/appsettings.json

```
{  "Logging": { "LogLevel": { "Default": "Information",  
"Microsoft.AspNetCore": "Warning" } },  "AllowedHosts": "*",  
"UploadSettings": { "AvatarDirectory": "wwwroot/uploads/avatars",  
"MaxFileSizeMB": 5 },  "Cors": { "AllowedOrigins":  
["http://localhost:3000"] } }
```

Section 3

Documentation

README — Quick Start

Stack: Next.js 14 + TypeScript + Tailwind CSS frontend, ASP.NET Core 8 C# backend.

Run Commands

Service	Commands	URL
Backend	<code>cd backend → dotnet restore → dotnet run</code>	<code>http://localhost:5000</code>
Frontend	<code>cd frontend → npm install → npm run dev</code>	<code>http://localhost:3000</code>
Swagger	Backend must be running	<code>http://localhost:5000/swagger</code>

Features

- Full CRUD for family members
- Drag-and-drop avatar upload (5 MB limit)
- Search and filter by name, email, or relationship
- Stats bar (total members, relationships, avatars)
- Age calculation from date of birth
- Grouped family view by relationship
- Toast notifications for all actions
- Keyboard accessibility and Escape key modal close
- Responsive design across screen sizes

Production Notes

For a real database, add EF Core + SQL Server. Replace `FamilyRepository` with an EF Core `DbContext` implementation. Register as `AddScoped` instead of `AddSingleton`.

API Endpoints Summary

Method	Endpoint	Description	Response
GET	<code>/api/family</code>	List all members (sorted by last name)	200 + array

Method	Endpoint	Description	Response
GET	/api/family/{id}	Get one member by GUID	200 or 404
POST	/api/family	Create member (requires firstName, lastName, relationship)	201 + created
PUT	/api/family/{id}	Full update of existing member	200 or 404
DELETE	/api/family/{id}	Delete member by GUID	204 or 404
POST	/api/upload/avatar	Multipart file upload, returns { url }	200 + url

Architecture Notes

- **Repository pattern** isolates all data access behind `IFamilyRepository`, making it straightforward to swap the in-memory store for a database.
- **Next.js API routes** proxy all calls to the C# backend, keeping CORS configuration simple and centralised.
- **Avatar uploads** are handled in Next.js via `formidable`, stored in `public/uploads/`; only the resulting URL string is persisted in the C# backend.
- **DTOs** (`CreateFamilyMemberRequest` / `UpdateFamilyMemberRequest`) are kept separate from the entity model to prevent over-posting attacks.
- **Thread safety** in `FamilyRepository` is maintained with a `lock` object around all mutations to the in-memory list.